Avaliação de Desempenho Computacional
Algoritmo Genético – Etapa 1

Felipe Avencourt Soares – 342904
Leonardo Rocha dos Santos – 341831

Instituto de Informática – UFRGS

2026/2

Ideia central

Proposta

Caracterizar o desempenho computacional de um Algoritmo Genético sequencial ao variar o tamanho da população e o tamanho dos cromossomos.

Não iremos:

- provar que o AG encontra soluções melhores;
- comparar versões sofisticadas de Algoritmos Genéticos;

Foco do trabalho

O AG se trata apenas de uma carga de trabalho computacional controlável.

O que é um Algoritmo Genético?

Um **Algoritmo Genético (AG)** é uma técnica de busca que mantém várias soluções candidatas ao mesmo tempo.

Ideia intuitiva

Em vez de testar uma solução por vez, o algoritmo mantém uma população de soluções e tenta melhorá-la geração após geração.

Lógica geral:

- avalia as soluções atuais;
- escolhe as melhores como base;
- combina e altera essas soluções;
- repete o processo.

Importante

O AG se baseia em vetores, números aleatórios e laços de repetição. São apenas operações computacionais inspiradas na evolução biológica.

Conceitos principais de um AG

Indivíduo	Uma solução candidata para o problema.
Cromossomo	A forma como o indivíduo é armazenado no programa. Neste trabalho, será um vetor binário.
População	Conjunto de N indivíduos mantidos simultaneamente.
Fitness	Valor numérico que mede a qualidade de um indivíduo.
Geração	Uma iteração completa: avaliar, selecionar, cruzar, mutar e criar nova população.

Exemplo

O cromossomo `101101` representa um indivíduo binário com $L = 6$.

Representação usada no Trabalho

Cada indivíduo será um vetor de bits com comprimento L.

$$\underbrace{[1, 0, 1, 1, 0, 1]}_{\text{um indivíduo}} \qquad L = 6$$

A população contém N indivíduos:

$$\begin{matrix} 101101 \\ 001011 \\ 111000 \\ \vdots \\ 010110 \end{matrix} \left.\begin{matrix} \\ \\ \\ \\ \\ \end{matrix}\right\} \begin{matrix} N \text{ linhas} \\ L \text{ bits por linha} \end{matrix}$$

Relação com desempenho: aumentar N ou L aumenta dados manipulados e trabalho por geração.

Fitness OneMax: cálculo da aptidão

A função de fitness escolhida será **OneMax**.

Ela conta quantos bits iguais a 1 existem no cromossomo.

$$x = [1, 0, 1, 1, 0, 1]$$

$$\mathit{fitness}(x) = 1 + 0 + 1 + 1 + 0 + 1 = 4$$

Por que usar OneMax?

É simples, controlada e fácil de verificar. Assim, o problema de otimização não esconde o comportamento computacional do AG.

Etapas de uma geração

Em cada geração, o AG executa as seguintes etapas:

1. **Avaliação**: calcula o fitness de cada indivíduo.
2. **Seleção**: escolhe indivíduos para serem pais.
3. **Crossover**: combina partes dos pais para gerar filhos.
4. **Mutação**: altera alguns bits dos filhos.
5. **Substituição**: a nova população passa a ser usada na próxima geração.

População atual $\rightarrow$ avaliação $\rightarrow$ seleção $\rightarrow$ crossover $\rightarrow$ mutação $\rightarrow$ nova população

No experimento

O número de gerações G será fixo para que todas as configurações executem uma quantidade comparável de trabalho.

Seleção por torneio

A seleção define quais indivíduos serão usados como pais.

Torneio de tamanho 2

Escolhem-se dois indivíduos aleatoriamente. O que tiver maior fitness vence e é selecionado como pai.

$$\begin{aligned} A &= 101101 \quad \mathit{fitness}(A) = 4 \\ B &= 001011 \quad \mathit{fitness}(B) = 3 \end{aligned}$$

$$\mathit{fitness}(A) > \mathit{fitness}(B) \quad \Rightarrow \quad A \text{ vence o torneio}$$

Efeito computacional: acessa valores de fitness e realiza comparações.

Crossover: combinando dois pais

O crossover gera novos indivíduos combinando partes de dois pais.

Crossover de um ponto

Escolhe-se uma posição de corte. O filho recebe a primeira parte de um pai e a segunda parte do outro.

$$\begin{array}{rcl} \textit{Pai}\ 1 & = & 101 \mid 101 \\ \textit{Pai}\ 2 & = & 001 \mid 011 \\ \hline \textit{Filho} & = & 101 \mid 011 \end{array}$$

Efeito computacional: copia e combina posições do cromossomo; custo cresce com L.

Mutação: pequenas alterações aleatórias

A mutação altera alguns bits dos filhos após o crossover.

Mutação bit-flip

Cada posição pode ser invertida: $0 \to 1$ ou $1 \to 0$.

$$101011 \xrightarrow{\text{inverte a posição 4}} 101111$$

Efeito computacional

A mutação envolve percorrer cromossomos e gerar decisões aleatórias. Mesmo sendo conceitualmente simples, pode ter custo relevante.

Lógica computacional do AG

Para uma população com N indivíduos e cromossomos de tamanho L:

- a avaliação percorre aproximadamente $N \times L$ bits por geração;
- crossover e mutação também manipulam cromossomos de tamanho L;
- com G gerações, a carga nominal cresce com $N \times L \times G$.

$$\text{carga nominal} \approx N \times L \times G$$

Exemplo

Se $N = 1000$, $L = 256$ e $G = 100$, então $NLG = 25.600.000$ posições de cromossomo são consideradas na normalização.

Objeto computacional

AG geracional sequencial

- Implementação em C++
- Execução em Linux
- Uma única thread
- Número fixo de gerações G

Configuração simplificada

- Fitness OneMax
- Seleção por torneio
- Crossover de um ponto
- Mutação bit-flip

Delimitação

A complexidade do trabalho está na avaliação de desempenho computacional, não na sofisticação do Algoritmo Genético.

Questão de pesquisa

Pergunta principal

Como o aumento de N e L afeta o desempenho computacional de um Algoritmo Genético sequencial?

Pergunta operacional:

- O custo cresce proporcionalmente a $N \times L$?
- Ou surgem regimes diferentes quando o conjunto de trabalho aumenta?

Hipótese

Para populações pequenas, o custo normalizado tenderá a ser estável. Para populações maiores, pode haver algumas degradações associadas à hierarquia de memória.

Fatores experimentais

Símbolo	Fator	Significado computacional
N	Tamanho da população	Número de indivíduos processados por geração
L	Comprimento do cromossomo	Número de posições binárias por indivíduo

$$\text{carga nominal por execução} \approx N \times L \times G$$

Controle experimental

Taxa de mutação, probabilidade de crossover, método de seleção e número de gerações serão mantidos constantes.

Técnica de avaliação

Técnica escolhida: medição em sistema real.

Ambiente experimental

- Ubuntu 24.04.4 LTS / kernel 7.0.0-30-generic
- Intel Core i7-1355U (2 P-cores + 8 E-cores)
- RAM: 15 GiB
- Caches: L1 privada por núcleo; L2 privada/compartilhada conforme o tipo de núcleo; L3 12 MiB compartilhada
- C++ com GCC 13.3.0
- Compilação: otimização -O3
- Execução em uma única thread, fixada em um núcleo

Procedimento

- Repetir cada configuração (N, L).
- Registrar sementes pseudoaleatórias.
- Manter hardware, software e flags fixos.

Ferramentas previstas

- Relógio monotônico: tempos.
- `perf stat`: contadores de hardware.
- `/usr/bin/time -v`: pico de memória RAM.

Métricas primárias

Tempo por geração

$$T_g = \frac{T}{G}$$

- Unidade: ms/geração
- Lower is better

Vazão normalizada

$$Q = \frac{NLG}{T}$$

- Posições de cromossomo/s
- Higher is better

Interpretação

Se Q cair conforme N e L aumentam, cada unidade nominal de trabalho está ficando mais cara.

Métricas explicativas

Contadores de hardware

- Ciclos de CPU
- Instruções executadas
- IPC
- Referências e faltas de cache
- Pico de memória RAM

$$IPC = \frac{\text{instruções}}{\text{ciclos}}$$

Decomposição do tempo

- Avaliação da aptidão
- Seleção
- Crossover
- Mutação

$$P_{\text{fase}} = \frac{T_{\text{fase}}}{T_{\text{total}}} \times 100\%$$

Percentual do tempo gasto em cada etapa.

Registro das medições

Uma linha por execução em arquivo CSV, contendo N, L, G, semente, número da repetição e todas as métricas coletadas.

Resultado esperado

Nosso estudo não pretende simplesmente concluir que cargas maiores demoram mais.

- Identificar se há mudança de regime no desempenho.
- Medir se o custo por unidade de trabalho permanece estável.
- Relacionar degradações a eventos de hardware quando possível.
- Descobrir qual fase do AG domina o tempo de execução.

Mensagem principal

O objetivo é explicar o comportamento computacional do AG, não avaliar sua eficácia como algoritmo de otimização, pois este não é o foco da disciplina.

Resumo da Etapa 1

Objeto computacional	AG sequencial simples, binário, com OneMax, medido sob variação de N e L.
Método de análise	Medição em sistema real Linux, com instrumentação interna e contadores de hardware.
Métricas	Tempo por geração, vazão normalizada, ciclos, instruções, IPC, cache misses, memória e tempo por fase.

Por que nossa escolha se justifica?

O AG fornece uma carga computacional parametrizável. Ao alterar N e L, podemos estudar escala dos dados, custo de execução e gargalos da máquina.